

به نام خدا



درس سیستم عامل
نیم سال اول ۰۵-۰۴
استاد: دکتر اسدی

دانشکده مهندسی کامپیوتر

تمرین سری هشتم

- پرسش‌های خود را در سامانه CW و تالار مربوط به تمرین مطرح نمایید.
- پاسخ سوالات را تایپ نمایید.
- اسکرین‌شات‌ها، عکس‌ها، فایل‌های مربوط به سوال عملی، گزارش تمرینات عملی و PDF قسمت تئوری را در پوشه با نام به فرمت HWNUM_StudentID1_StudentID2 ذخیره نمایید. سپس آن را zip نمایید و در صفحه درس بارگذاری نمایید. به عنوان مثال یک فایل بارگذاری شده قابل قبول باید دارای فرمت HW1_400123456_403123456.zip باشد.
- هر دانشجو می‌تواند حداکثر سه تمرین را با دو روز تأخیر بدون کاهش نمره ارسال نماید.
- تمرینات عملی و تئوری به صورت گروه‌های دو نفر تحویل داده شود.
- هر دو عضو گروه موظف هستند تمرینات خود را بارگذاری کنند.
- عواقب عدم تطابق بین پاسخ دو عضو گروه برعهده خودشان است.
- تحویل تمرین به صورت انگلیسی مجاز نیست. در صورت تحویل تمرین به صورت انگلیسی (حتی بخشی از تمرین) نمره تمرین موردنظر صفر در نظر گرفته می‌شود.
- در صورت مشاهده تقلب برای بار اول نمره هر دو طرف صفر می‌شود. در صورت تکرار نمره کل تمرینات صفر خواهد شد.
- استفاده از ابزارهایی مانند ChatGPT به منظور ابزار کمک آموزشی مجاز است به شرط آن که به خروجی آن اکتفا نشود.
- توجه شود که پروژه نهایی درس در گروه‌های چهار نفر تحویل گرفته می‌شود.
- سوالات با عنوان اختیاری نمره‌ای ندارند اما جواب دادن به آن‌ها کمک به سزایی در یادگیری درس می‌کند.
- هیچ تمرینی خارج از CW تصحیح نخواهد شد. لذا توصیه می‌شود حداقل یک الی دو ساعت قبل از زمان پایان تمرین برای بارگذاری تمرین اقدام شود.

تمارین تئوری

۱. برنامه زیر را در نظر بگیرید و با توجه به آن به سوالات پاسخ دهید:

```

1      P1: {                                P2: {
2          shared int x;                    shared int x;
3          x = 10;                          x = 10;
4          while (1) {                      while (1) {
5              x = x - 1;                    x = x - 1;
6              x = x + 1;                    x = x + 1;
7              if (x != 10)                  if (x != 10)
8                  printf("x is %d",x);      printf("x is %d",x);
9          }                                }
10     }                                    }

```

(آ) دنباله‌ای از ترتیب اجرای دستورات ارائه دهید به طوری که عبارت "x is 10" چاپ شود.

(ب) دنباله‌ای از ترتیب اجرای دستورات ارائه دهید به طوری که عبارت "x is 8" چاپ شود.

(ج) نشان دهید که در کجا و چگونه باید سمافورها را به برنامه اضافه کرد تا اطمینان حاصل شود که دستور printf() هرگز اجرا نمی‌شود. راه حل شما باید حداکثر میزان همزمانی^۱ ممکن را فراهم کند.

۲. یک فرآیند را در نظر بگیرید که در آن چندین ریسمان^۲ یک ساختار داده مشترک از نوع Last-In-First-Out (LIFO) را به اشتراک می‌گذارند. این ساختار داده یک لیست پیوندی^۳ از عناصر struct node است و یک اشاره گر به نام top که به عنصر بالای لیست اشاره می‌کند، میان همه‌ی نخ‌ها مشترک است.

برای اضافه کردن یک عنصر به لیست (push)، یک نخ به صورت پویا^۴ برای یک struct node در heap حافظه تخصیص می‌دهد و سپس اشاره گر این ساختار را به صورت زیر در ساختار داده قرار می‌دهد:

```

1 void push(struct node *n) {
2     n->next = top;
3     top = n;
4 }

```

نخی که قصد دارد یک عنصر را از ساختار داده حذف کند (pop)، کد زیر را اجرا می‌کند:

```

1 struct node *pop(void) {
2     struct node *result = top;
3     if (result != NULL)
4         top = result->next;
5     return result;
6 }

```

بر این اساس به سوالات زیر پاسخ دهید:

(آ) آیا ممکن است زمانی که دو نخ به طور همزمان بخواهند دو عنصر را به این ساختار داده اضافه کنند، رقابت^۵ رخ دهد؟ اگر پاسخ مثبت است، ترتیب دقیق اجرای دستورات در نخ‌ها که منجر به وقوع رقابت می‌شود را توصیف کنید. در هر مرحله توضیح دهید که شکل ساختار داده به چه صورتی خواهد بود.

¹concurrency

²thread

³linked list

⁴dynamic

⁵race condition

(ب) آیا ممکن است زمانی که دو نخ به طور همزمان بخواهند دو عنصر را از این ساختار داده حذف کنند، رقابت رخ دهد؟ اگر پاسخ مثبت است، ترتیب دقیق اجرای دستورات در نخ‌ها که منجر به وقوع رقابت می‌شود را توصیف کنید. در هر مرحله توضیح دهید که شکل ساختار داده به چه صورتی خواهد بود.

(ج) توضیح دهید که چگونه با استفاده از lock می‌توان از ایجاد رقابت در این ساختار داده جلوگیری کرد.

۳. فرض کنید تخصیص دهنده حافظه چندریسمانی زیر را داریم که به طور تقریبی به شکل زیر ترسیم شده است.

```

1 int bytes_left = MAX_HEAP_SIZE;
2 pthread_cond_t c;
3 pthread_mutex_t m;
4
5 void *allocate(int size) {
6     pthread_mutex_lock(&m);
7     while (bytes_left < size)
8         pthread_cond_wait(&c, &m);
9     void *ptr = ...; // get mem from internal data structs
10    bytes_left -= size;
11    pthread_mutex_unlock(&m);
12    return ptr;
13 }
14 void free(void *ptr, int size) {
15     pthread_mutex_lock(&m);
16     bytes_left += size;
17     pthread_cond_signal(&c);
18     pthread_mutex_unlock(&m);
19 }

```

فرض کنید تمام حافظه مصرف شده است (یعنی مقدار bytes_left برابر با ۰ است). سپس:

- ریسمان اول (T1) تابع allocate(100) را فراخوانی می‌کند.
- مدتی بعد، ریسمان دوم (T2) تابع allocate(1000) را فراخوانی می‌کند.
- در نهایت، مدتی بعد، ریسمان سوم (T3) تابع free(200) را فراخوانی می‌کند.

با فرض اینکه تمام فراخوانی‌های مربوط به توابع کتابخانه نخ‌ها طبق انتظار به درستی کار می‌کنند، توضیح دهید پس از اجرای توالی فراخوانی‌ها، چه وضعیت‌هایی برای نخ‌های T1، T2 و T3 ممکن یا غیرممکن است. برای هر حالت، دلیل خود را به طور مختصر بیان کنید.

۴. سیستمی را در نظر بگیرید که دارای دو نخ با زمان‌بندی پیش‌دستانه^۶ است. یکی از نخ‌ها تابع WriteA را اجرا می‌کند که در زیر نشان داده شده است. نخ دیگر تابع WriteB را اجرا می‌کند که آن هم در زیر آمده است.

هر دو تابع از kprintf برای تولید خروجی روی کنسول استفاده می‌کنند. تابع random که توسط WriteA فراخوانی می‌شود، یک عدد صحیح غیرمنفی تصادفی برمی‌گرداند. توابع WriteA و WriteB با استفاده از دو سمافور Sa و Sb همگام‌سازی شده‌اند. مقدار اولیه هر دو سمافور صفر است.

فرض کنید که هر فراخوانی مجزای kprintf به صورت atomic اجرا می‌شود.

```

1 void WriteA() {
2     unsigned int n,i;
3     while(1) {
4         n = random();
5         for(i=0;i<n;i++)

```

^۶preemptive

```

6         kprintf('A');
7         for(i=0;i<n;i++)
8             V(Sb);
9         for(i=0;i<n;i++)
10            P(Sa);
11     }
12 }
13
14 void WriteB() {
15     while(1) {
16         P(Sb);
17         kprintf('B');
18         V(Sa);
19     }
20 }

```

(آ) کدام یک از این پیشوندها ممکن است توسط اجرای همزمان این دو نخ در این سیستم تولید شده باشند؟

- ABABABABAB •
- BABABABABA •
- AAAAAAAAAA •
- AAABABABAB •
- AAABBBBAAB •
- AAAAAABBBB •
- AAABBBAAAB •
- BBBBBAAAAB •

(ب) فرض کنید مقدار اولیه سمافور Sb به جای ۰، برابر با ۱ باشد. یک پیشوند خروجی ۱۰ نویسه‌ای از کنسول نشان دهید که در این حالت می‌تواند تولید شود، اما در صورتی که مقدار اولیه Sb برابر با ۰ باشد، امکان تولید آن وجود نداشته باشد.

۵. الگوریتم زیر را برای حل مسئله‌ی انحصار متقابل n ریسمان در نظر بگیرید. فرض می‌کنیم که ریسمان‌ها، هر کدام یک id مشخص و غیرتکراری داشته باشد و با متغیر i به آن دسترسی دارند. مقدار اولیه‌ی $turn$ ، در ابتدای اجرای برنامه می‌تواند هر مقدار دلخواهی از مقادیر ۱ تا n باشد. به ازای هر ریسمان i ، یک متغیر $flag[i]$ با مقدار اولیه‌ی ۰ تعیین شده است. فرض می‌کنیم که متغیرها به صورت globally تعریف شده‌اند و همه‌ی ریسمان‌ها به آن‌ها دسترسی دارند. هر ریسمان، کد زیر را اجرا می‌کند:

```

1 while (true) {
2   L:  flag[i] = 1;
3       while (turn != i) {
4         if (flag[turn] == 0) {
5           turn = i;
6         }
7       }
8       flag[i] = 2;
9       for (int j = 1 ; j <= n ; j++) {
10        if (j != i && flag[j] == 2) {
11          goto L;
12        }
13      }
14      // critical section
15      flag[i] = 0;
16 }

```

با توجه به کد بالا، به پرسش‌های زیر پاسخ دهید:

(آ) آیا کد مربوطه، مسئله‌ی انحصار متقابل را حل می‌کند؟ اگر پاسخ شما مثبت است، به کمک برهان خلف نشان دهید که هیچ دو ریسمانی به طور همزمان در ناحیه‌ی بحرانی قرار نمی‌گیرند. اگر پاسخ شما منفی است، سناریویی را توضیح دهید که دو ریسمان به طور همزمان وارد این ناحیه شوند.

(ب) در کد مربوطه، از دستور `flag[i] = 2` تا ابتدای ورود ریسمان به ناحیه‌ی بحرانی را حذف کنید. بار دیگر به پرسش الف پاسخ دهید.

۶. نسخه اصلاح‌شده زیر از الگوریتم Peterson را در نظر بگیرید که یک برنامه‌نویس در آن خطا کرده است:

```
1 // Shared variables
2 boolean flag[2] = {false, false};
3 int turn = 0;
4
5 // Process Pi (i = 0 or 1, j = 1-i)
6 do {
7     flag[i] = true;
8     while (flag[j] && turn == i); // ERROR: Should be turn == j
9
10    // Critical Section
11
12    flag[i] = false;
13    // Remainder Section
14 } while (true);
```

این پیاده‌سازی اشتباه را تحلیل کنید و به سوالات زیر پاسخ دهید:

(آ) به طور مشخص توضیح دهید که کدام یک از سه الزام بخش بحرانی (bounded, progress, mutual exclusion) (waiting) نقض می‌شوند.

(ب) یک سناریوی اجرای مشخص که نقض را نشان می‌دهد ارائه دهید.

(ج) توضیح دهید چرا نسخه صحیح (`turn == j`) از این نقض جلوگیری می‌کند؟

۷. (اختیاری) درباره‌ی سوالات زیر تحقیق کنید:

(آ) پیشوند^۷ LOCK در معماری اینتل چه کاری انجام می‌دهد؟ چگونه می‌توان به کمک آن `compare and swap` را پیاده‌سازی کرد؟

(ب) در معماری‌های RISC مانند ARM یا MIPS که از مدل `load-store` استفاده می‌کنند چه طور می‌توان عملیات `atomic` را پیاده‌سازی کرد؟

(ج) به نظر شما چه چالش‌هایی برای انجام دادن عملیات `atomic` برای اعداد ۶۴ بیتی در سیستم‌های ۳۲ بیتی وجود دارد؟ بررسی کنید که به عنوان مثال آیا می‌توان در پردازنده‌های ۳۲ بیتی اینتل عملیات `compare and swap` را برای دو عدد ۶۴ بیتی انجام داد؟ آیا همیشه و در همه‌ی معماری‌ها می‌توان این کار را به صورت `lock-free` انجام داد؟

(د) تابع `__kuser_cmpxchg64` در سیستم‌عامل لینوکس چه کاری انجام می‌دهد، چگونه پیاده‌سازی شده است و چه کمکی برای حل مشکلات قسمت قبل می‌کند؟

(ه) در عملیات `atomic`، Memory Ordering به چه معنا است؟

(و) فراخوانی سیستمی `futex` در سیستم‌عامل لینوکس چه کاری انجام می‌دهد؟

(ز) فراخوانی سیستمی `flock` در سیستم‌عامل لینوکس چه کاری انجام می‌دهد؟ یک نمونه از کاربرد این فراخوانی را مثال بزنید.

⁷Prefix

(ح) در زبان برنامه‌نویسی Golang به جای استفاده از Mutex استفاده از Channel پیشنهاد شده است. درباره‌ی Channel ها تحقیق کنید. مزیت و معایب آن‌ها را نسبت به Mutex ها بیان کنید.

۸. (اختیاری) مفهوم تراکنش دهه‌ها است که در پایگاه داده‌ها^۸ برای مدیریت دسترسی همزمان به داده‌های مشترک با موفقیت استفاده می‌شود. با این حال، در سیستم عامل‌ها و برنامه‌نویسی موازی، برنامه‌نویسان به طور سنتی از قفل‌ها و سایر ابزارهای همگام‌سازی سطح پایین استفاده کرده‌اند. در این باره به سوالات زیر پاسخ دهید:

(آ) تراکنش^۹ به چه معنا است؟ یک نمونه از تراکنش‌ها در پایگاه‌های داده را مثال بزنید و ذکر کنید در صورت وجود نداشتن آن چه سختی پیش می‌آمد.

(ب) چرا مفهوم تراکنش در ابتدا برای عملیات حافظه به کار گرفته نشد؟ چه چالش‌هایی در پیاده‌سازی چنین مکانیزمی وجود دارد؟ مفهوم حافظه تراکنشی^{۱۰} را تحقیق کنید و توضیح دهید که چگونه این مفهوم به حل این چالش‌ها کمک می‌کند و چه مزایایی برای مدیریت دسترسی همزمان به حافظه مشترک ارائه می‌دهد.

(ج) دستور العمل‌های Intel-TSX و AMD-ASF در معماری x86 چه کاری را انجام می‌دهند و چه کمکی به برنامه‌نویسان می‌کنند؟

⁸Databases

⁹Transaction

¹⁰Transactional Memory

تمارین عملی

۱. XV6 OS

برای این تمرین از کد بارگذاری شده در سامانه CW استفاده کنید.

در xv6، متغیر سراسری^{۱۱} ticks یک شمارنده‌ی زمان سیستم است که نشان می‌دهد از زمان شروع سیستم چند timer interrupt رخ داده است. از این متغیر برای محاسبه‌ی زمان و sleep استفاده می‌شود.

در xv6 توابع sys_pause و sys_uptime مقدار متغیر سراسری ticks را می‌خوانند. از آنجایی که این متغیر ممکن است همزمان توسط تابع clockintr به‌روزرسانی شود، این دو تابع قبل از خواندن ticks، قفل چرخشی tickslock را می‌گیرند. این کار باعث می‌شود clockintr نتواند به صورت همزمان مقدار ticks را تغییر دهد اما این مورد یک مشکل دارد آن هم این که چند هسته به طور همزمان نمی‌توانند مقدار ticks را بخوانند در حالی که این کار ایمن است و مشکلی ندارد. در نتیجه استفاده از spinlock معمولی باعث کاهش کارایی می‌شود.

راه‌حلی که برای این مشکل وجود دارد استفاده از Read-Write Lock است. در این نوع قفل، دو نوع دارنده قفل (Reader و Writer) وجود دارد. در این نوع قفل، موارد زیر باید رعایت شوند:

- در هر زمان حداکثر یک نویسنده می‌تواند قفل را در اختیار داشته باشد.
- وقتی یک نویسنده قفل را گرفته است، هیچ خواننده‌ای اجازه دسترسی ندارد.
- اگر نویسنده‌ای وجود نداشته باشد، چندین خواننده می‌توانند همزمان قفل را بگیرند.

یکی از مشکلات این قفل این است که اگر تعداد زیادی خواننده وجود داشته باشد، ممکن است نویسنده هیچوقت نتواند قفل را بگیرد. یعنی خواننده‌ها مدام قفل را می‌گیرند و آزاد می‌کنند اما هیچگاه تعداد خواننده‌ها به صفر نمی‌رسد تا نویسنده بتواند قفل را بگیرد و بنویسد. برای حل این مشکل معمولاً از Writer Priority استفاده می‌شود که به این صورت است که اگر یک نویسنده تلاش کرد که قفل را بگیرد، خواننده‌های جدید دیگر اجازه ورود ندارند تا زمانی که نویسنده قفل را بگیرد و سپس آن را آزاد کند. (البته دقت کنید که نویسنده باید منتظر بماند تا خواننده‌های فعلی قفل را آزاد کنند) به طور کلی در این تمرین قصد داریم تا rwspinlock را در xv6 پیاده‌سازی کنیم. جزئیات پیاده‌سازی باید مطابق با توضیحات بالا باشد و تمامی موارد رعایت شوند.

برای انجام این تمرین شما نیاز است تا TODOهای درون فایل kernel/spinlock.c را تکمیل کنید. همچنین نیاز است تا تعریف struct rwspinlock را نیز در فایل kernel/spinlock.h تغییر دهید. دقت کنید که مجاز هستید تا توابعی را اضافه کنید و یا تغییر دهید.

گام‌های پیشنهادی:

۱. ابتدا نگاهی به تابع sys_rwlktest در فایل kernel/spinlock.c بیندازید. این تابع همان آزمون نهایی شما است.
۲. شما باید فراخوانی‌های acquire() و release() در توابع read_acquire_inner، write_acquire_inner، read_release_inner و write_release_inner را با پیاده‌سازی قفل خودتان جایگزین کنید.
۳. مراقب ترتیب‌های مختلف اجرای همزمان باشید. به طور مثال اگر یک خواننده بررسی می‌کند که نویسنده‌ای در انتظار نیست، چگونه مطمئن می‌شود که این وضعیت قبل از گرفتن قفل تغییر نکرده است.
۴. مطالعه توابع atomic داخلی gcc می‌تواند به شما کمک کند. آن‌ها را در این لینک می‌توانید مشاهده کنید.

در انتهای پیاده‌سازی لازم است تا تغییرات خود را با استفاده از آزمون rwlktest مورد بررسی قرار دهید. خروجی مورد نظر به صورت زیر خواهد بود:

```
1 $ rwlktest
2 rwspinlock_test: step 1
3 rwspinlock_test: step 2
```

¹¹global

```

4 rwspinlock_test: step 3
5 rwspinlock_test: step 4
6 rwspinlock_test: step 5
7 rwspinlock_test: step 6
8 rwspinlock_test: step 7
9 rwspinlock_test(0): 0
10 rwspinlock_test(2): 0
11 rwspinlock_test(1): 0
12 rwlkttest: passed 3/3
13 $

```

همچنین باید برنامه زیر را نیز در سیستم عامل اجرا کنید و تمامی آزمون‌های آن به درستی اجرا شود.

```
1 usertests -q
```

۲. Docker

این تمرین در ادامه پیاده‌سازی شما از تمرین ۶ است. بنابراین نیاز به دریافت تگ جدیدی از مخزن تمرین نیست. در این تمرین قصد داریم با دانش بدست آمده در تمرین ۷ در رابطه با OverlayFS، یک کانتینر را براساس یک ایمیج استاندارد داکر اجرا کنیم. با این تفاوت که این بار ایمیج بر اساس لایه‌های آن ایجاد می‌شود و به صورت مستقیم تمام محتوای آن مورد استفاده قرار نمی‌گیرد. در نهایت یک مثال عملی از شبکه در داکر انجام خواهیم داد.

گام‌های تمرین

آ. ایمیج پایتون با ورژن زیر را دریافت کنید.

```
1 docker pull docker.arvancloud.ir/python:3.11
```

اگر ایمیج را از قبل روی ماشین خود نداشته باشید، باید دریافت آن را به صورت لایه‌لایه مشاهده کنید.

ب. به واسطه دستور زیر مسیری که در آن تغییرات لایه آخر وجود دارد را مشاهده کنید.

```
1 docker inspect docker.arvancloud.ir/python:3.11 | jq '.[].GraphDriver.Data.UpperDir'
```

به این مسیر بروید و در پوشه بالایی این مسیر فایل lower را بررسی کنید. این فایل حاوی اطلاعات لایه‌های زیرین این ایمیج است. اما اگر توجه کنید، فرمت نام لایه‌ها متفاوت از ID بدست آمده از خروجی دستوری است که اجرا کردید.

این فرمت نام‌گذاری در واقع نشان‌دهنده یک symlink به لایه‌های زیرین است. برای مثال یکی از لایه‌ها را با فرمت مشابه فرمت زیر بررسی کنید.

```
1 sudo ls -alh /var/lib/docker/overlay2/1/BQY5KHJ2NDWSRQTIJLUYZQH2VF
```

این فایل در واقع یک symlink به محتوای پوشه diff از لایه مد نظر است. بنابراین برای اجرای یک کانتینر براساس این ایمیج باید تمام این لایه‌های موجود در فایل lower، به همراه پوشه diff در لایه آخر، همگی با حفظ ترتیب به عنوان lower در کانتینر در حال اجرا قرار بگیرند.

ج. به برنامه خود یک پرچم base-image- اضافه کنید و تغییرات لازم برای اجرای کانتینر را بدهید.

د. محتویات فایل setup.c و خصوصاً تابع setup_container_dir را به نحوه‌ای تغییر دهید تا یک OverlayFS براساس لایه‌های ایمیج مشخص شده ایجاد کند. در نهایت پوشه merged از فایل سیستم ایجاد شده به عنوان ریشه یا root کانتینر مد نظر باید استفاده شود.

توجه کنید که فایل سیستم کانتینر حتماً باید به صورت لایه‌لایه ساخته شود و ساختاری مانند ساختار تمرین ۶ پذیرفته نیست.

سوال: مزیت این روش نسبت به پیاده‌سازی تمرین ۶ چیست؟

ه. سوال: در رابطه با دستور docker commit و کاربردهای آن تحقیق کنید. همچنین توضیح دهید چگونه می‌توان این دستور را با الهام از معماری لایه‌لایه در ایمج‌ها، این دستور را پیاده‌سازی کرد؟
 اختیاری: دستور `docker commit <container-name>` را به پیاده‌سازی خود اضافه کنید.
 سوال: براساس درکی که در انجام این تمرینات بدست آورده‌اید، دو مفهوم ایمج و کانتینر را به لحاظ شباهت‌ها و تفاوت‌ها مقایسه کنید.

موارد لازم جهت تحویل

- پاسخ به سه سوال.
- توضیح مختصر از هر کدام از گام‌های تمرین.
- اجرای یک برنامه ساده فیبوناچی در پایتون داخل کانتینر به جهت نشان دادن عملکرد درست کانتینر.
- کدهای تغییر داده شده.

۳. توسعه سیستم مدیریت پرونده

در تمارین گذشته، یک سامانه مدیریت پرونده تک پرداز و تک رشته‌ای درست کردیم. در این تمرین تلاش می‌کنیم این محدودیت‌ها را رفع کنیم.

گام‌های تمرین

۱. مشکلات احتمالی در مورد استفاده همزمان چند کاربر از این سامانه را نام ببرید.
 ۲. تلاش کنید یکی از مثال‌هایی که در بخش قبل نام بردید را روی سامانه خود اجرا کنید و نتایج آن را گزارش کنید.
 ۳. روی بخش‌های مختلف سامانه خود قفل بگذارید به طوری که ۲ کاربر به طور همزمان بتوانند بدون ایجاد مشکل روی سامانه از آن استفاده کنند.
 ۴. نقاط ضعف این قفل‌ها را بررسی کنید و آن‌ها را با مثال‌های عملی نشان دهید.
- توجه کنید که برای نشان دادن نقاط ضعف، می‌توانید عده‌ای ثابت و یا زمان‌بندی‌ها را دستکاری کنید تا این مشکلات آشکار شوند.